

Weekly Rapport — Weeks 8–9

Real-Time Anomaly Detection Platform for Branch Teller Operations
Yassine — Amen Bank, Direction Centrale du Système d'Information
June 30 – July 9, 2026

Contents

- 1 Executive Summary
- 2 Week Summary
- 3 Architectural Refactor: Core Extraction
 - 3.1 Problem Statement
 - 3.2 Design Evaluation
 - 3.3 Pattern Applied
 - 3.4 Project Structure
- 4 Simulation API — FastAPI
 - 4.1 Architecture
 - 4.2 Implementation
 - 4.3 Design Decisions
 - 4.4 Validation
- 5 Health Monitoring and Observability
 - 5.1 Design Rationale
 - 5.2 Two-Signal Architecture
 - 5.3 Pushgateway Evaluation
 - 5.4 ServiceMetrics Class
 - 5.5 Instrumentation Pattern
- 6 Full Docker Compose Orchestration
 - 6.1 Container Inventory
 - 6.2 Startup Sequencing
 - 6.3 Operational Workflow
 - 6.4 PostgreSQL Integration
- 7 Key Technical Learnings
- 8 Artifacts Produced
- 9 Next Steps

1 Executive Summary

This two-week period transformed the project from a collection of validated components into a fully containerized, observable production system. Four major milestones were achieved: (1) a clean architectural refactor separating core logic from transport, (2) a FastAPI simulation API enabling synchronous event scoring, (3) a comprehensive health monitoring layer with Prometheus instrumentation, and (4) full Docker Compose orchestration of 12 services with automated startup sequencing. The entire system now starts with a single `docker compose up -d` command.

2 Week Summary

Date	Phase	Key Deliverable
June 30 (Mon.)	E2E pipeline completion	Cold-start buffer seeding, weeks 6–7 rapport
July 3 (Thu.)	Core extraction refactor	EnrichmentCore / ScoringCore / DecisionCore with DI
July 4 (Fri.)	Simulation API + containerization	FastAPI /score endpoint, Dockerfile.api, Docker Compose
July 9 (Wed.)	Health monitoring + full orchestration	Prometheus metrics, Kafka exporter, 12-service Docker Compose

3 Architectural Refactor: Core Extraction

3.1 Problem Statement

The supervisor requested a FastAPI REST endpoint for simulation mode, but all business logic was tightly coupled to Kafka consumer/producer wrappers. A synchronous /score endpoint cannot route through the asynchronous Kafka pipeline without unacceptable complexity.

3.2 Design Evaluation

Three options were evaluated for the synchronous scoring endpoint:

Option	Description	Verdict
Request-reply via Kafka	Push to raw-events, correlate on decisions	Rejected: latency, complexity
Bypass Kafka	Duplicate logic or call services directly	Rejected: tight coupling, duplication
BFF/Orchestration	Import core modules, call enrich → score → decide in-process	Selected

3.3 Pattern Applied

Each of the three services was split into two layers:

Core module (EnrichmentCore, ScoringCore, DecisionCore): receives external dependencies (Redis, model objects) via dependency injection, exposes a single public method (facade pattern), uses private helpers for sub-steps. No Kafka dependency.

Streaming wrapper (EnrichmentStreamingService, etc.): thin Kafka consumer/producer plumbing. Receives a core instance via DI. The `run()` poll loop calls the core's public method and produces the

result. No Redis, no models, no feature computation.

The `__main__` block is the sole wiring point: creates Redis client, model objects, core instance, and streaming wrapper, then starts the loop.

3.4 Project Structure After Refactor

```
src/  
  enrichment/      # EnrichmentCore (core logic)  
  scoring/         # ScoringCore + AEScorer + LSTMScorer + ScoreFusion  
  decision/        # DecisionCore + DecisionService engine  
  streaming/       # Thin Kafka wrappers (3 services)  
  api/            # FastAPI simulation API  
  monitoring/      # ServiceMetrics (Prometheus instrumentation)  
  batch/          # hydrate_redis.py  
  config.py       # Centralized env-based configuration
```

4 Simulation API — FastAPI

4.1 Architecture

The simulation API implements the BFF/orchestration pattern: it imports `EnrichmentCore`, `ScoringCore`, and `DecisionCore` directly, wires them with shared Redis and model dependencies at startup, and exposes a synchronous `/score` endpoint.

4.2 Implementation

Three-file structure:

- **src/api/models.py** — Pydantic request models with cross-field validation enforcing payload–operation_type consistency (e.g., a *virement* must include `beneficiary_id` in its payload)
- **src/api/routes.py** — Single `/score` POST endpoint: `enrich` → `score` → `decide`, returns full decision output
- **src/api/app.py** — Lifespan context manager for startup wiring, `app.state` for DI of shared core instances

4.3 Design Decisions

- **dry_run flag on `EnrichmentCore.enrich_event()`**: simulation mode sets `dry_run=True` to skip Redis buffer/sequence updates, ensuring simulation requests don't pollute the streaming pipeline's state. A simple boolean flag was chosen over a split compute/commit pattern to avoid accidental streaming pipeline breakage.
- **CPU-only PyTorch in Docker**: avoids 700MB+ of CUDA libraries in the image
- **Model files mounted as runtime volume** (`./models:/app/models`): supports weekly retraining without image rebuilds

4.4 Validation

Tested end-to-end both locally and in Docker:

- 15,000 DT *versement* → BLOCK with BCT regulatory flag (exceeds 10,000 DT reporting threshold)
- 9,500 DT *virement* to a new beneficiary → BLOCK (smurfing band + unknown counterparty)

5 Health Monitoring and Observability

5.1 Design Rationale

A streaming consumer can be 'alive' (process running) but completely useless — stalled polling, Redis unreachable, model inference failing silently. Binary health checks are insufficient; the system needs quantitative observability across three failure categories:

- **Ingestion failures** — consumer stops polling or falls behind
- **Processing failures** — Redis timeout, model exception, malformed event
- **Output failures** — cannot produce to output topic

5.2 Two-Signal Architecture

External signal — Consumer lag: A Kafka exporter container connects to RedPanda and exposes `kafka_consumergroup_lag` metrics per consumer group, topic, and partition. This covers ingestion and output health without any service-level code changes. Prometheus scrapes the exporter on port 9308.

Internal signal — Processing metrics: Only the service itself knows whether a Redis call failed or a model threw an exception. Each core module is instrumented with Prometheus counters and histograms, injected via DI alongside Redis and model dependencies.

5.3 Pushgateway Evaluation

Pushgateway was evaluated and rejected for three reasons: (1) it was designed for short-lived batch jobs, not long-lived consumers; (2) if the consumer process dies, the last-pushed metrics remain stale — Pushgateway cannot distinguish 'healthy with no new errors' from 'process is dead'; (3) metric collisions when scaling to multiple instances. The embedded `/metrics` endpoint approach naturally provides an `up=0` signal when Prometheus cannot scrape a dead process.

5.4 ServiceMetrics Class

A shared `src/monitoring/metrics.py` module provides a `ServiceMetrics` class injected into all three cores:

Metric	Type	Labels	Scope
<code>{service}_events_processed_total</code>	Counter	<code>operation_type</code>	All services
<code>{service}_errors_total</code>	Counter	<code>error_class</code>	All services
<code>{service}_processing_duration_seconds</code>	Histogram	—	All services
<code>scoring_fused_score_distribution</code>	Histogram	—	ScoringCore only

The `fused_score_distribution` histogram uses buckets dense near the alert thresholds (0.95 / 0.99 / 0.999) to enable model drift detection by comparing current score distributions against the reference distribution saved during training.

5.5 Instrumentation Pattern

Metrics are injected via DI (`metrics=None` default preserves backward compatibility). The core's public method wraps execution in a `try/except/finally` block:

- **try:** process event, increment events_processed on success
- **except:** classify exception (redis.RedisError → redis_failure, else → logic_error), increment errors counter, re-raise
- **finally:** observe processing_duration (always recorded, even on failure)

time.monotonic() is used instead of time.time() to avoid clock adjustment artifacts.

6 Full Docker Compose Orchestration

6.1 Container Inventory

Container	Image	Port	Role
redpanda	redpandadata/redpanda	9092	Message broker (Kafka-compatible)
redis	redis:7-alpine	6379	Hot profiles, buffers, sequences
postgres	postgres:16-alpine	5432	Cold store (clients, transactions, snapshots)
simulation-api	Dockerfile.api	8000	REST /score endpoint
enrichment-service	Dockerfile.api	9100	Streaming: raw → enriched
scoring-service	Dockerfile.api	9101	Streaming: enriched → scored
decision-service	Dockerfile.api	9102	Streaming: scored → decisions
kafka-exporter	danielqsj/kafka-exporter	9308	Consumer lag → Prometheus
prometheus	prom/prometheus	9090	Metrics collection + storage
hydrate-redis	Dockerfile.api	—	Batch: profile hydration (runs at startup)
db-seed	Dockerfile.api	—	One-time: PostgreSQL data loading
simulator	Dockerfile.api	—	Bank system stand-in (on-demand)

6.2 Startup Sequencing

Docker Compose v2 condition directives enforce correct ordering:

```
postgres (healthy) → hydrate-redis (completed_successfully) → streaming services (started)
```

hydrate-redis runs automatically on every docker compose up, ensuring Redis is always warm before the pipeline starts. It is idempotent — writing the same profiles twice is harmless. The db-seed job uses a profiles: [seed] flag and only runs on first-time setup. The simulator uses profiles: [simulate] and runs on-demand.

6.3 Operational Workflow

```
# First time only:
docker compose --profile seed run --rm db-seed

# Every time:
docker compose up -d

# Push events through pipeline:
docker compose --profile simulate up simulator
```

6.4 PostgreSQL Integration

- Schema DDL (config/init.sql) auto-executed via /docker-entrypoint-initdb.d/ on first container creation
- load_postgresql.py updated to read DB config from src.config (environment variables) instead of hardcoded credentials

- Health check (pg_isready) gates all downstream dependencies

7 Key Technical Learnings

Principle	Context
BFF over request-reply	FastAPI imports core modules directly rather than routing through Kafka — avoids duplication and tight coupling
Instrumentation is not business logic	Prometheus counters/histograms are injected via DI like any other dependency; the core uses them but does not own them
Separate failure domains	Health reporting (Prometheus scrape) must not share infrastructure (Redis) with the thing being monitored
Consumer lag as heartbeat	The pipeline's own output is its health signal — no additional heartbeat mechanism needed for throughput monitoring
Pushgateway anti-pattern	Pushgateway masks process death for long-lived services; embedded /metrics provides natural up=0 on crash
service_completed_successfully	Docker Compose v2 enables job-then-service ordering without external orchestration

8 Artifacts Produced

Artifact	Path
ServiceMetrics class	src/monitoring/metrics.py
Prometheus config	config/prometheus.yml
PostgreSQL DDL	config/init.sql
Bootstrap script (Windows)	scripts/bootstrap.ps1
Instrumented cores	src/enrichment/core.py, src/scoring/core.py, src/decision/core.py
Updated streaming wrappers	src/streaming/enrichment_service.py, scoring_service.py, decision_service.py
Full Docker Compose	docker-compose.yml (12 services)
Updated Dockerfile	Dockerfile.api (added scripts/, config/)

9 Next Steps

- **Streamlit dashboard:** consumes the decisions topic, displays alert list with SHAP explanations, pipeline health from Prometheus, KPIs per branch/operation, simulation mode
- **MLflow integration:** model versioning, experiment tracking
- **Airflow DAGs:** orchestrated weekly retraining, batch profile rebuild, Redis hydration
- **Batch enrichment rewrite:** aligned with 30-feature contrast schema, triggered by Airflow
- **Grafana dashboards (optional):** operational monitoring complement to the supervisor-facing Streamlit dashboard